TANKSHIFT – FIRMWARE LOGIC

Part 9 of the Master Pack (Revised) – HT site version, ESP32

Team: [Name 1], [Name 2], [Name 3] – REVA University

Date: 27 October 2026

Status: Logic complete, ready to code. Written so a second-year student can follow it.

9.1 What the Firmware Must Do (Plain Words)

1. Never let the tank run dry. This beats every other rule.
2. Fill the tanks in the cheap window (Karnataka: 10 pm–6 am, Dec–Jun).
3. Never start the pump in the expensive windows (6–9 am, 6–10 pm) unless rule 1 forces it.
4. Keep pump starts away from the site's monthly demand peak.
5. If several pumps are on one controller, start them one at a time.
6. Measure every unit pumped and work out the rupees saved.
7. Keep working with no internet, no cloud and no phone.
8. If anything goes wrong, fall back to behaving like the float switch it replaced.

9.2 Inputs and Outputs

Signal	Type	Source	Used for
OHT_FULL	digital in	float, top of overhead tank	stop pumping
OHT_RESERVE	digital in	float, 30 % of overhead tank	emergency pump (rule 1)
OHT_LEVEL	0–100 %	JSN-SR04T ultrasonic	planning, logging
SUMP_HIGH	digital in	float, top of sump	sump is full
SUMP_LOW	digital in	float, bottom of sump	dry-run lockout
TIME	hh:mm, date	DS3231 RTC	tariff window, season
kW, kWh, A, V, PF	Modbus RTU	3-phase meter	savings, dry-run check, demand
TEST_BTN	digital in	push button	manual test / reset fault
PUMP_CMD	digital out	relay → starter coil	the only output that matters
STATUS_LED	digital out	LED	blink code (9.11)

9.3 Settings File (settings.json, stored on the ESP32)

All site-specific numbers live here, not in code. Changed from the phone app or by re-flashing the file.

```
{
  "site_id": "REVA-PH1",
  "state": "KA",
  "pumps": 1,
  "pump_kw": 11.0,
  "reserve_pct": 30,
  "target_full_pct": 95,
  "daily_use_litres": 60000,
  "oht_litres": 20000,
  "min_run_min": 5,
  "max_run_min": 120,
  "rest_between_runs_min": 10,
  "stagger_sec": 120,
  "demand_guard_kva": 950,
  "contract_demand_kva": 1000,
  "dry_run_amps_min": 4.0,
  "tariff_file": "tariff_KA_HT2.json",
  "fallback_mode": "float"
}
```

9.4 Tariff Table File (tariff_KA_HT2.json)

Downloadable file, not firmware. Same box works in another state by swapping this file. Values from the KERC Electricity Tariff-2026 ToD table for HT-2(a), (b), (c).

```
{
  "name": "Karnataka HT-2 ToD, KERC 2025-28",
  "months_active": [12, 1, 2, 3, 4, 5, 6],
  "windows": [
    { "from": "06:00", "to": "09:00", "adder_rs": 1.00, "label": "peak_morning" },
    { "from": "09:00", "to": "18:00", "adder_rs": 0.00, "label": "day" },
    { "from": "18:00", "to": "22:00", "adder_rs": 1.00, "label": "peak_evening" },
    { "from": "22:00", "to": "06:00", "adder_rs": -1.00, "label": "night_rebate" }
  ],
  "base_rate_rs": 7.00,
  "demand_charge_rs_per_kva": 380
}
```

Outside the active months (Jul–Nov) every adder is 0: the firmware then runs on demand-guard and float logic only.

9.5 State Machine

State	Pump	Enter when	Leave when
BOOT	off	power on	self-test passes → IDLE (≤ 30 s)
IDLE	off	nothing to do	any pump reason below, or fault
PUMP_EMERGENCY	on	OHT_RESERVE open (tank below 30 %)	OHT_FULL, or SUMP_LOW, or fault
PUMP_SCHEDULED	on	inside cheap window AND tank below target AND demand OK	OHT_FULL, window ends, SUMP_LOW, max_run_min, or fault
PUMP_TOPUP	on	last 60 min before an expensive window AND tank below target AND adder ≤ 0	OHT_FULL, window starts, or fault
WAIT_DEMAND	off	pump wanted but site kVA above demand_guard	kVA drops for 60 s, or OHT_RESERVE opens
LOCKOUT_PEAK	off	inside expensive window, tank above reserve	window ends, or OHT_RESERVE opens

State	Pump	Enter when	Leave when
DRY_RUN	off	SUMP_LOW open, or motor amps < dry_run_amps_min for 30 s while on	SUMP_HIGH closes, or TEST_BTN held 5 s
FAULT	off (relay released)	sensor disagreement, RTC lost, watchdog reset ×3	TEST_BTN held 5 s; starter is in float-fallback meanwhile

Priority if several states apply: FAULT > DRY_RUN > PUMP_EMERGENCY > LOCKOUT_PEAK > WAIT_DEMAND > PUMP_SCHEDULED > PUMP_TOPUP > IDLE.

9.6 Decision Loop (every 10 seconds)

```
loop every 10 s:
  read floats, ultrasonic, RTC, meter
  check_sensors()                  // float vs ultrasonic sanity
  if fault            -> FAULT
  if sump_low or dry  -> DRY_RUN
  if oht_reserve_open -> PUMP_EMERGENCY     // rule 1, always wins
  adder = tariff.adder_now(time, month)
  if adder > 0        -> LOCKOUT_PEAK
  want = oht_level < target_full_pct
  if want and site_kva > demand_guard_kva -> WAIT_DEMAND
  if want and adder < 0                   -> PUMP_SCHEDULED
  if want and adder == 0 and next_peak_in_min <= 60 -> PUMP_TOPUP
  else                                    -> IDLE
  apply_min_run_and_rest()        // no chattering
  apply_stagger_if_multi_pump()
  set_relay(state is a PUMP_* state)
  log_sample()
  kick_watchdog()
```

Why these rules: the night window is 8 hours long and the OHT holds about a third of a day, so one full fill at night plus a top-up before the evening peak covers a normal campus day. The emergency rule covers everything else.

9.7 Hard-Coded Safety Rules (not in settings, cannot be switched off)

9. OHT_RESERVE open → pump ON, regardless of tariff, demand or schedule.
10. SUMP_LOW open → pump OFF, regardless of everything, until SUMP_HIGH closes.
11. OHT_FULL closed → pump OFF within 2 seconds.
12. Motor current below dry_run_amps_min for 30 s while running → DRY_RUN.
13. OHT_FULL and OHT_RESERVE both open for more than 5 s is impossible → FAULT.
14. Ultrasonic says > 90 % but OHT_RESERVE float says empty → trust the float, flag FAULT.
15. Relay pin is pulled low at boot; the relay module is HIGH-trigger; pump can never twitch on reset.
16. Any FAULT releases the relay; the starter's own float circuit keeps water flowing.

9.8 Demand-Peak Guard

HT billing demand is the highest kVA in the month. A pump starting on top of the campus peak adds its kVA to the bill for the whole month at ₹350–380/kVA. The controller reads site kVA from the main meter (if available over Modbus) or estimates it from time-of-day history.

- demand_guard_kva is set 5 % below the contract demand or 5 % below last month's peak, whichever is lower.

- If site kVA > guard, the pump waits (WAIT_DEMAND) up to 60 min, then re-checks.
- Emergency fill ignores the guard: water beats money.
- The controller stores the hour of each month's peak and avoids scheduled starts inside ±30 min of it.

9.9 Multi-Pump Stagger

If one controller runs 2–3 pumps (borewell + transfer), they never start together. Each start waits stagger_sec (default 120 s) after the previous one. Running order: transfer pump first (it empties the sump), then borewells. Combined running kW is capped so the night fill does not create a new night demand peak.

9.10 Logging and Savings Calculation

Field	Every	Note
timestamp, state, oht_pct, sump_hi/lo	10 s	ring buffer in RAM, 1-min averages to flash
kWh, kW, A, V, PF	1 min	from the meter
window label, adder	1 min	from tariff file
units_in_window[label]	daily	4 counters per day
rs_saved	daily	see formula
peak_kva_avoided	monthly	pump kVA × (1 if start avoided peak hour)

Savings formula (per day):

```
baseline_units = units pumped today (the pump would have run anyway)
baseline_cost  = Σ over windows of baseline share × (base + adder)
                 using the site's pre-TankShift window shares
                 (from the last 6 bills, entered in settings)
actual_cost    = Σ over windows of units_in_window × (base + adder)
rs_saved       = baseline_cost - actual_cost
demand_saved   = (peak_kva_avoided) × demand_charge_rs_per_kva  (monthly)
```

Flash holds 90 days at 1-min resolution (about 13 MB on a 16 MB ESP32 or on SD). Upload to the cloud when Wi-Fi/4G is present; nothing is lost if it is not.

9.11 Watchdog, Faults and Offline Behaviour

- Hardware watchdog (ESP32 task WDT) at 30 s. Three resets in an hour → FAULT and stop retrying.
- RTC lost (DS3231 oscillator-stop flag) → no tariff decisions; run on float logic only; LED fault code; keep logging with uptime counter.
- Meter not answering for 5 min → keep pumping decisions, mark savings as "unmetered", LED warning.
- Wi-Fi/4G down → nothing changes; the box is designed to run offline for months.
- Power cut → on return, BOOT self-test, read RTC, resume. No stored pump command survives a reboot.

LED blink codes: solid = pump on; 1 blink/2 s = idle; 2 blinks = waiting (demand or peak lockout); 3 blinks = dry-run; fast blink = FAULT; off = no power to logic.

9.12 Bench Test Checklist (before any site)

17. Lamp on the relay output, not a pump.
18. Simulate each float with a toggle switch; confirm every row of the 9.5 table.
19. Set RTC to 21:59 and watch the lamp come on at 22:00 with tank below target.
20. Set RTC to 05:59, lamp off at 06:00 even with tank at 50 %.
21. Open OHT_RESERVE at 07:00: lamp must come on within 10 s.
22. Open SUMP_LOW while running: lamp off within 2 s.
23. Pull the ultrasonic cable: box keeps working on floats, LED warning.
24. Pull the RTC: box falls to float-only, fault code.
25. Pull power during pumping and restore: relay stays off through boot.
26. Leave running 48 h on the bench; check the log has no gaps.

9.13 ESP32 Code Skeleton (Arduino framework)

Enough to start. Fill the TODOs; the logic above maps one-to-one.

```
#include <Wire.h>
#include <RTClib.h>          // DS3231
#include <ModbusMaster.h>    // meter over MAX485
#include <ArduinoJson.h>
#include <LittleFS.h>
#include <esp_task_wdt.h>

// ---- pins (Part 6) ----
#define PIN_RELAY   26
#define PIN_LED     14
#define PIN_BTN      4
#define PIN_OHT_FULL 32
#define PIN_OHT_RES  33
#define PIN_SUMP_LOW 34
#define PIN_SUMP_HI  35
#define PIN_US_TRIG  25
#define PIN_US_ECHO  27
#define PIN_RS485_DE  5

enum State { BOOT, IDLE, PUMP_EMERGENCY, PUMP_SCHEDULED, PUMP_TOPUP,
             WAIT_DEMAND, LOCKOUT_PEAK, DRY_RUN, FAULT };
State state = BOOT;

RTC_DS3231 rtc;
ModbusMaster meter;
StaticJsonDocument<2048> cfg, tariff;

struct Inputs { bool ohtFull, ohtRes, sumpLow, sumpHi; float ohtPct;
                float kw, kva, amps; DateTime now; bool rtcOk, meterOk; } in;

void preTx()  { digitalWrite(PIN_RS485_DE, HIGH); }
void postTx() { digitalWrite(PIN_RS485_DE, LOW);  }

void setup() {
  pinMode(PIN_RELAY, OUTPUT); digitalWrite(PIN_RELAY, LOW);   // pump off first
  pinMode(PIN_LED, OUTPUT);
  pinMode(PIN_BTN, INPUT_PULLUP);
  pinMode(PIN_OHT_FULL, INPUT_PULLUP); pinMode(PIN_OHT_RES, INPUT_PULLUP);
  pinMode(PIN_SUMP_LOW, INPUT); pinMode(PIN_SUMP_HI, INPUT);  // ext. pull-ups
  pinMode(PIN_US_TRIG, OUTPUT); pinMode(PIN_US_ECHO, INPUT);
  pinMode(PIN_RS485_DE, OUTPUT); postTx();

  Serial2.begin(9600, SERIAL_8N1, 16, 17);
```

```
  meter.begin(1, Serial2); meter.preTransmission(preTx);
meter.postTransmission(postTx);
  Wire.begin(21, 22); in.rtcOk = rtc.begin() && !rtc.lostPower();

  LittleFS.begin(true);
  loadJson("/settings.json", cfg);
  loadJson(cfg["tariff_file"] | "/tariff_KA_HT2.json", tariff);

  esp_task_wdt_init(30, true); esp_task_wdt_add(NULL);
  state = selfTest() ? IDLE : FAULT;
}

void loop() {
  static uint32_t last = 0;
  if (millis() - last >= 10000) {           // 10 s tick
    last = millis();
    readInputs();
    state = decide();                        // 9.6, with 9.7 inside
    applyRelay(isPumpState(state));
    logSample();
    esp_task_wdt_reset();
  }
  blinkLed(state);
  handleButton();                            // 5 s hold clears DRY_RUN / FAULT
}

float adderNow() {
  if (!in.rtcOk) return 0;                   // no clock: no tariff decisions
  int m = in.now.month(); bool active = false;
  for (int x : tariff["months_active"].as<JsonArray>()) if (x == m) active = true;
  if (!active) return 0;
  int hm = in.now.hour()*60 + in.now.minute();
  for (JsonObject w : tariff["windows"].as<JsonArray>()) {
    int f = toMin(w["from"]), t = toMin(w["to"]);
    bool inside = (f < t) ? (hm >= f && hm < t) : (hm >= f || hm < t);  // wraps
midnight
    if (inside) return w["adder_rs"];
  }
  return 0;
}

State decide() {
  if (!sensorsAgree())            return FAULT;          // 9.7 rule 5, 6
  if (in.sumpLow || dryRunAmps()) return DRY_RUN;        // rule 2, 4
  if (in.ohtFull)                 return IDLE;           // rule 3
  if (in.ohtRes)                  return PUMP_EMERGENCY; // rule 1
  float adder = adderNow();
  bool want = in.ohtPct < (cfg["target_full_pct"] | 95);
  if (adder > 0)                  return LOCKOUT_PEAK;
  if (want && in.kva > (cfg["demand_guard_kva"] | 1e9)) return WAIT_DEMAND;
  if (want && adder < 0)          return PUMP_SCHEDULED;
  if (want && minutesToNextPeak() <= 60) return PUMP_TOPUP;
  return IDLE;
}

// TODO: readInputs(), sensorsAgree(), dryRunAmps(), minutesToNextPeak(),
//       applyRelay() with min_run / rest / stagger, logSample() to LittleFS,
//       blinkLed(), handleButton(), selfTest(), loadJson(), toMin().
```

Libraries: RTClib (Adafruit), ModbusMaster (4-20ma), ArduinoJson (Benoit Blanchon), LittleFS (built in). Meter register map: Eastron SDM630 – voltage 0x0000, current 0x0006, active power 0x000C, apparent power 0x0012, total kWh 0x0156 (all float32, input registers).

9.14 Phone App (minimum version for the pilot)

Not needed for the 90-day pilot to work. A web page served by the ESP32 on the campus Wi-Fi is enough: shows state, tank %, today's units by window, rupees saved this month, and lets the estate team edit three settings (reserve %, target %, demand guard). Everything else is read-only. Dashboard for the treasurer is Step 2 of the grant.